

# LispのREPLと四則演算

式を読み、計算し、結果を返す

情報システム学科 3年・選択・2単位

# 今日の到達目標

- S式をトークンと木で表せる
- 四則演算の評価規則を説明できる
- エラー後も使えるREPLを実装できる

# 前回との接続

電卓では、演算子の優先順位を構文で表しました。

Lispでは括弧がまとまりを指定し、先頭に演算子を書きます。

`1 + 2 * 3` と `(+ 1 (* 2 3))` を比較します。

# 教材の言語仕様

本教材では、Pythonをホストにする小さなLisp方言を設計します。

これは教材上の選択であり、Lisp処理系や正式シラバスの共通仕様ではありません。

第9回の範囲は、数値、S式、四則演算、REPLです。

# 最初の実演

```
python3 examples/09-lisp-arithmetic/lisp.py --eval "(+ 1 (* 2 3))"
```

期待する値は 7 です。

数値の表示規則も、実装の契約としてそろえます。

# S式の形

```
42  
(+ 1 2)  
(* (+ 1 2) 4)
```

数値はそのまま値になります。リストは先頭と残りに分けます。

# 読取と評価

| 段階   | 仕事            |
|------|---------------|
| 字句解析 | 括弧・記号・数値を切り出す |
| 読取   | 入れ子のリストを組み立てる |
| 評価   | 式から値を計算する     |
| 表示   | 値を文字列にする      |

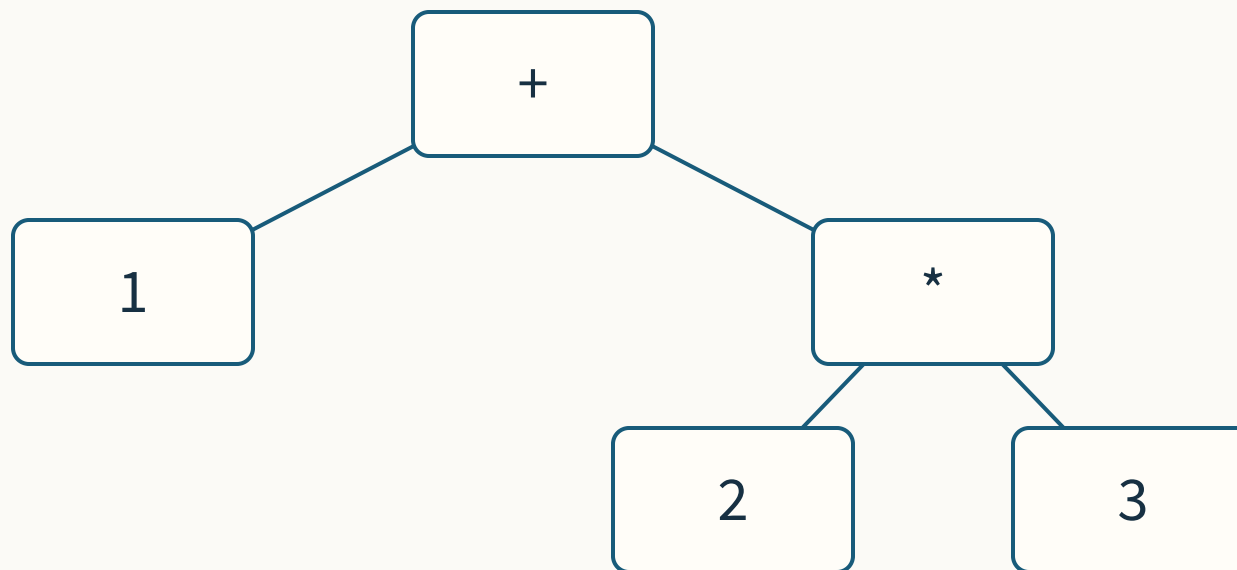
# トークン列

入力: (+ 1 (\* 2 3))

トークン: ( + 1 ( \* 2 3 ) )

区切りの括弧は捨てず、トークンとして残します。

# 構文木



内側の式の値を、外側の引数に使用します。

# 読取の再帰

( を読んだら、対応する ) まで要素を読みます。

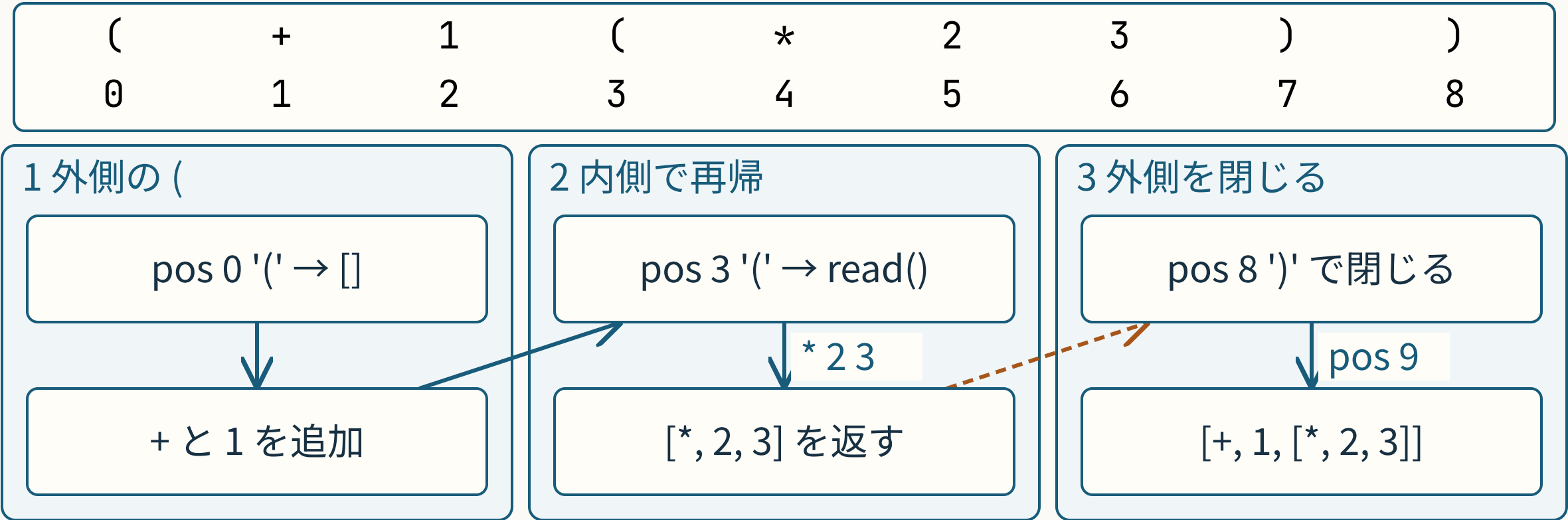
要素が再び ( なら、同じ処理を呼びます。

入力を消費する位置と、返す木を分けて考えます。

内側の (\* 2 3) を読み終えた直後に、pos が指すトークンを予想してください。

# read の再帰と pos

( を読むと read が自分を呼び、内側の ) で戻ってから外側を続けます。



別入力: (+ 1 2)) で、余分な ) のエラーが示す offset を予想してから実行します。

# 数値の評価

```
def evaluate_number(node):  
    return node
```

この断片は評価規則の説明用です。実行可能な完成版は例題ディレクトリにあります。

# 演算の評価

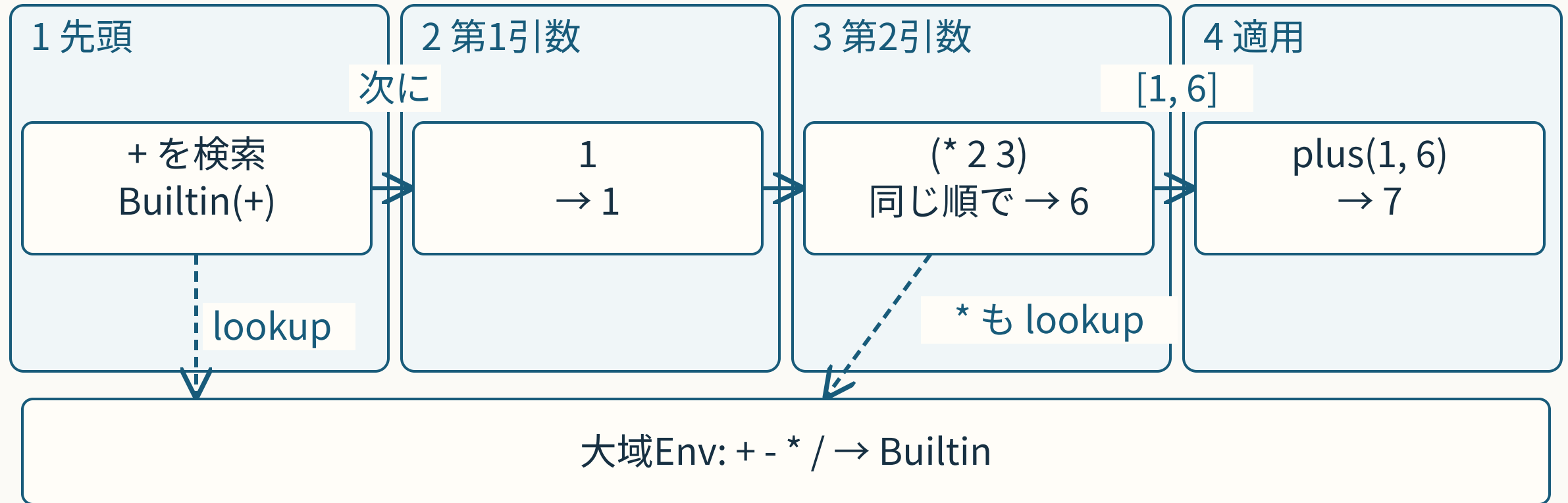
1. 先頭の演算子を確認する
2. 各引数を評価する
3. 評価済みの値で計算する

`(+ 1 (* 2 3))` の `*` を先に計算する理由を述べてください。

`+` の検索と `(* 2 3)` の計算は、どちらが先に起こるか予想してください。

# 先頭→引数→適用の順

先頭を環境で探し、引数を左から評価してから Builtin を呼びびます。



別入力: (- 10 2 3) の評価順と値を同じ4コマで書き、実行の 5 と照合します。

# 四則演算の契約

演算子ごとに、引数個数と型の扱いを決めます。

今回の受入例では、各演算子に2個の数値を渡します。

単項演算や可変長引数を追加する場合は、参照実装の仕様を先に確認してください。

# 可変長の四則

参照版の `+` と `*` は0個以上、`-` と `/` は1個以上の引数を取ります。

```
(+ 1 2 3)
(- 10 2 3)
(/ 12 2 3)
```

結果は `6`、`5`、`2` です。減算と除算は左から順に計算します。

# ゼロ除算

( / 8 0 ) は構文として読めます。

評価時に失敗したことを利用者に伝えます。

入力全体を「構文エラー」と呼ぶと、原因を探しにくくなります。

# 括弧の不足

```
(+ 1 2  
(+ 1 2))
```

1行目は閉じ括弧の不足、2行目は余分な閉じ括弧です。  
どの段階が検出するかを予測してください。

# REPLの循環

| 文字    | 役割       |
|-------|----------|
| Read  | 入力をS式にする |
| Eval  | 値を求める    |
| Print | 値を表示する   |
| Loop  | 次の入力待つ   |

# REPLの実演

```
python3 examples/09-lisp-arithmetic/lisp.py
```

(+ 2 3)、 (/ 1 0)、 (\* 4 5) の順に入力してください。

最後の式で 20 が得られます。終了は :quit またはEOFです。

# エラーからの回復

1回の入力の失敗は、その入力の結果として報告します。

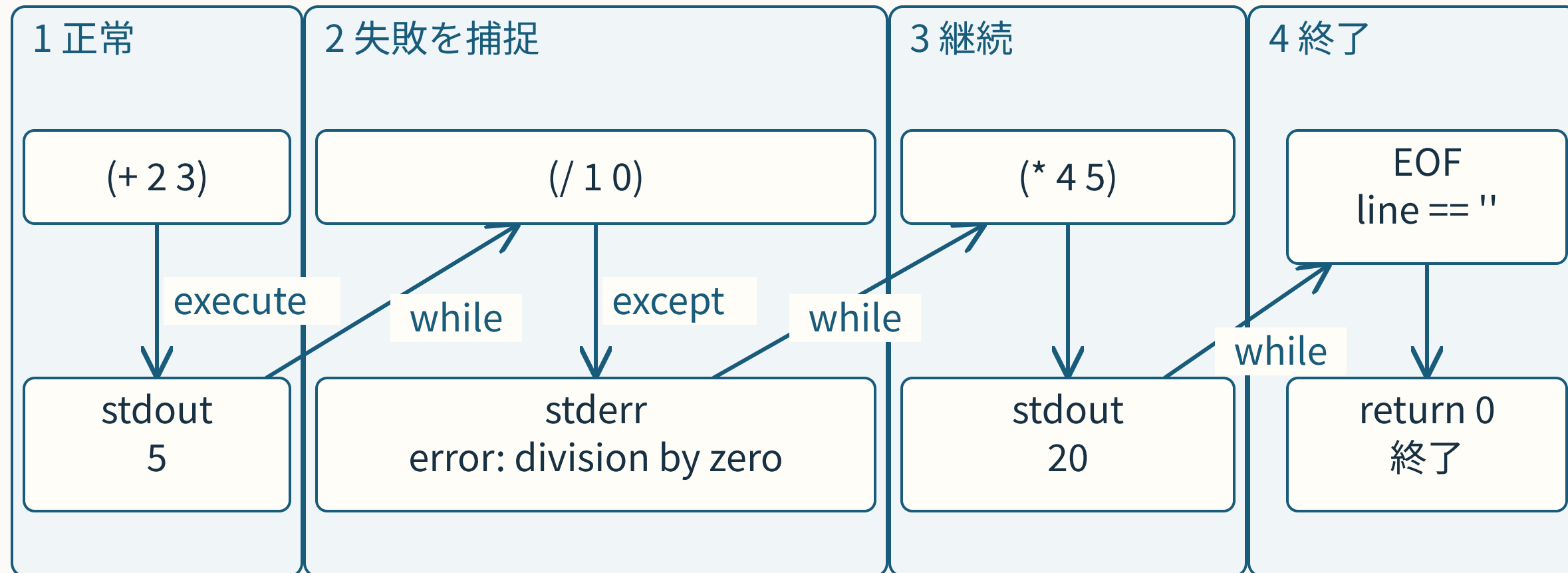
ループの外まで例外を逃がすと、次の入力を受けられません。

想定する例外は扱い、実装バグの情報は残します。

( / 1 0 ) で例外が起きた後、処理がどこへ戻るか予想してください。

# try はループの内側

try は while の内側にあり、失敗はその1行で終わって次の行へ進みます。



別入力: `(+ 1 2` の後に `(* 4 5)` を入力し、読取の失敗でも続くか確かめます。

# 演習1 読取の観察

`( * ( + 2 3 ) ( - 9 4 ) )` を手でトークン列にしてください。

次に入れ子のリストを書き、実装の読取結果と照合してください。

検証用の入力を1個、自分で追加してください。

# 演習2 評価器

2引数の四則演算を実装してください。

(+ 2 3)、(- 9 4)、(\* 3 4)、(/ 8 2) を確かめてください。

入れ子の式も、同じ評価関数で扱います。

# 演習3 REPL

正常入力の中にエラー入力を挟んでください。

結果、診断、次の結果を実行ログに残してください。

EOFで自然に終了できることも確認してください。

# 確認問い

(+ 1 2) の括弧を消すと、どの規則が失われるか。

読取器と評価器を1つの関数にすると、何の検証が難しくなるか。

# 提出物の証拠

演習成果物の案は `exercises/09.md` にあります。

コードに加え、入れ子の正常例とエラー後の回復例を残してください。

提出先・締切などの運用は、授業の案内で確認してください。

# まとめと次回

括弧で木を作り、再帰で値を求めました。

REPLは、入力ごとに失敗を区切ります。

次回は、名前を値へ結び付け、関数を定義します。

# 参考

- 本教材: `examples/09-lisp-arithmetic/`
- Python 3.12 [エラーと例外](#)
- SICP [評価器](#)

外部資料の言語仕様と、教材方言との差に注意してください。

自習用の [検証済みサンプルとテスト](#) も公開しています。